

```
playerMove = getPlayerMove(playerTurn, gameBoard)
gameBoard[playerMove] = playerTurn
```

Each iteration of this loop consists of a single turn. First, we display the game board to the player. Second, the player selects a column to drop a tile in, and third, we update the game board data structure.

Next, we evaluate the results of the player's move:

```
# Check for a win or tie:
if isWinner(playerTurn, gameBoard):
    displayBoard(gameBoard) # Display the board one last time.
    print("Player {} has won!".format(playerTurn))
    sys.exit()
elif isFull(gameBoard):
    displayBoard(gameBoard) # Display the board one last time.
    print("There is a tie!")
    sys.exit()
```

If the player made a winning move, `isWinner()` returns `True` and the game ends. If the player filled the board and there is no winner, `isFull()` returns `True` and the game ends. Note that instead of calling `sys.exit()`, we could have used a simple `break` statement. This would have caused the execution to break out of the `while` loop, and because there is no code in the `main()` function after this loop, the function would return to the `main()` call at the bottom of the program, causing the program to end. But I opted to use `sys.exit()` to make it clear to programmers reading the code that the program will immediately terminate.

If the game hasn't ended, the following lines set `playerTurn` to the other player:

```
# Switch turns to other player:
if playerTurn == PLAYER_X:
    playerTurn = PLAYER_O
elif playerTurn == PLAYER_O:
    playerTurn = PLAYER_X
```

Notice that I could have made the `elif` statement into a simple `else` statement without a condition. But recall the Zen of Python tenet that *explicit is better than implicit*. This code *explicitly* says that if it's player